区块链原理与技术实验

第八次实验报告

模拟实现比特币 P2P 网络

多矿工出块、区块广播与 ACK 回传

课程 / 实验	区块链原理与技术实验 / 第八次实验
姓名	_____
学号	_____
完成日期	2026 年 11 月 12 日

本实验在基础 P2P 模拟程序上扩展两个矿工。两个矿工生成带矿工编号的唯一区块，区块在邻居网络中去重转发，ACK 按 origin 返回；每个节点还保存已知区块并按父区块关系保留分叉。

一、实验目的

模拟比特币底层 P2P 网络中的区块传播过程，理解节点邻接关系、洪泛式转发、消息去重和确认回传。重点实现多个矿工并行出块，验证区块身份唯一，并让确认消息回到真正的出块矿工。

二、实验环境

项目	配置
运行环境	Windows，Python 3.12
启动方式	python p2pSim.py 4
网络规模	4 个本地节点；Node 0、Node 1 为矿工；Node 2、Node 3 为普通节点
主要模块	socket、threading、json、random、secrets

三、实验原理与总体设计

节点通过 TCP 连接发送 JSON 消息。矿工构造区块并将消息发给各自邻居；邻居第一次收到某个 block_id 时，将其写入本地区块集合并继续转发。seen_blocks 以区块 ID 去重，阻断循环广播。

区块携带 origin 字段标明出块节点。首次接收区块的节点向 ipTable[origin] 发送 ACK；矿工仅处理 origin 等于自身节点号的 ACK。区块 ID 采用 block_<矿工编号>_<本矿工序号>，避免多个矿工计数器相同造成误判。

选做部分以字典保存每个节点已知的全部区块，以 prev_hash 记录父区块。分叉区块分别保留，不以到达先后覆盖；新块基于当前已知的最高链末端生成，高度相同时使用区块 ID 作确定性选择。

四、关键实现

4.1 多矿工出块与唯一标识

MINER_IDS 设置为 [0, 1]。两个矿工各自维护 block_counter，并在区块 ID 中写入 node_id，因此 block_0_1 与 block_1_1 是不同区块。每个新区块同时写入本地 blockchain 和 seen_blocks 后再广播，避免矿工收到网络回传时重复并入。

4.2 区块 origin 与 ACK 路由

Block 类新增 origin 字段，并在 to_dict/from_dict 中序列化。接收节点读取 origin 后直接连接该矿工的 IP/端口发送 ACK；ACK 消息也包含 origin。服务端根据当前节点号匹配 origin，只由目标矿工打印并处理该确认。

4.3 本地链与分叉保留

每个节点使用 block_id 到 Block 的映射保存全部已知区块，chain_lock 保护接收线程和挖矿线程对本地区块数据的访问。父区块关系通过 prev_hash 保存，因此两个矿工若基于同一个父块出块，会形成两个并列分支；best_tip 选出当前最高分支的末端供后续新区块引用。

五、运行步骤

1. 在实验目录打开命令行。
2. 执行 python p2pSim.py 4。
3. 观察 Node 0 与 Node 1 分别广播 block_0_n、block_1_n。
4. 观察其他节点收到并转发区块，以及 ACK 分别返回 Miner 0、Miner 1。
5. 按 Ctrl+C 停止程序。

六、运行结果与验证

本次以 4 个节点启动程序。运行日志显示两个矿工都广播了区块；Node 2、Node 3 收到并转发区块；针对 block_0_1 的 ACK 返回 Miner 0，针对 block_1_1 的 ACK 返回 Miner 1。block_0_1 与 block_1_1 均以创世标记 0 为 prev_hash，形成并列分叉。



图 1 四节点、双矿工运行日志关键片段（源自本次程序实际输出）

验证点	观察依据	结果
多个矿工出块	出现 block_0_1、block_1_1，origin 分别为 0、1	通过
全局唯一与去重	不同矿工 ID 不同；同一 ID 经网络转发仅首次接收时继续传播	通过
ACK 正确路由	block_0_1 的 ACK 到 Miner 0；block_1_1 的 ACK 到 Miner 1	通过
本地区块链与分叉	block_0_1 与 block_1_1 共享创世父块，并在各节点保留	通过

七、实验总结

实验完成了双矿工并行出块、全局唯一区块标识、origin 定向 ACK 和本地区块集合维护。运行结果验证了新区块能够在节点网络中传播，重复消息由区块 ID 过滤，确认能够返回对应矿工；共享创世父块的并发区块也能以两条分支保留。

本程序用于教学模拟：prev_hash 使用区块 ID 简化表示，未实现真实哈希、共识难度、交易验证及持久化。